

VestigeKV — Defense Document

Anticipated questions, with the detail the whitepaper compresses out

Each section below is one anticipated question. The whitepaper carries the compressed statement; this document carries the full argument and the measured numbers behind it.

1 Q1: At a 99.8% trigger rate, is tier 1 still necessary at long context? Does recall depend on it?

Yes, and recall is structurally defined in terms of it.

Trigger frequency is not workload share. At $S=512k$ a step attends $\approx 16,384$ tier-1 rows and fetches ≈ 120 recalled ones (0.024% of the archive): tier 1 supplies over 99% of the attended rows and essentially all of the attention mass; recall is the patch, not the payload. The correct mental model is a cache hierarchy — tier 1 is the *hit path* (query-independent, prefetched at prefill, riding the fixed in-graph channel at zero per-step cost), recall is the *miss path* (query-dependent, riding the variable channel). “At least one miss on 99.8% of steps, at 0.024% miss volume” describes a healthy cache, not a redundant one. Removing tier 1 forces every relevant row through the variable scan-and-fetch path every step — the fired set balloons from ~ 120 rows to the entire important set, converting cheap fixed reads into scanned fetches in every regime.

Recall depends on tier 1 three times over. (a) The fire criterion is $\text{score}_a > \max_1$, and $\max_1$ is tier 1’s content — without the kept set the competition standard does not exist. (b) The entropy gate is computed over the kept-tier softmax. (c) Calibration defines a hard query as one whose target is *archived*, i.e. relative to the tier-1 partition. The counterfactual was run and rejected in the pre-registered record: trigger designs without the kept-max competition either recalled poorly ($\text{recall@4} = 0.489$) or fired on 99.5% of rows — it is precisely the competition against tier 1’s optimum that tames firing from 99.5% to 0.024%. Remove tier 1 and the mechanism collapses back to the similarity top- k shape this design supersedes.

2 Q2: How does this differ from ArkVale?

ArkVale (Chen et al., 2024) is the closest prior design — evict, summarize, recall on demand — and a port team should know exactly where this tier departs from it:

1. **Unit and summary.** ArkVale summarizes evicted *pages* with lossy bounding digests and recalls whole pages. This tier indexes individual *rows* with an exact 64-dim branch summand, a rank- r content sketch, and a scalar residual certificate; recalled rows are bit-exact (no lossy digest exists anywhere, per the no-quantization rule).
2. **Why digests cannot be tight under RoPE.** A page digest must cover its keys; rotation spreads a page’s keys across phases, so the cover is structurally loose — measured digest layer-uniformity 0.672 (NoPE) vs. 0.024 (RoPE) in the reference record. ArkVale operates in the

loose regime; this tier is built for the NoPE regime where per-row summaries are meaningful at all.

3. **Certified competition vs. similarity scoring.** ArkVale admits pages by digest-similarity ranking. Here a row fires only when its certified score beats the query’s own kept-tier maximum, with z self-calibrated to a 90% recall target from the prefix — an admission *criterion with a guarantee*, not a similarity heuristic.
4. **Eviction signal.** ArkVale’s eviction relies on query-history importance; tier-1 eviction here is query-independent (σ over the branch), so the whole partition exists before any query does — the setting ArkVale cannot serve.
5. **Scan economy (design intent, with a measured caveat).** The entropy gate is meant to skip whole scans when the kept tier is already confident, where ArkVale scans per step. On the **sclang** port the gate is *off* even after calibration is repaired — with 3–63 hard queries per layer the fitted threshold would pass more than 60% of queries, so the self-disable rule correctly declines to install a filter that filters nothing. The conformal z , which shares the same hard set, does calibrate. Budget a full scan per head per step; the gate is an optimization the port does not currently collect.

NoPE-specific optimizations in the recall path. Each departure is enabled by a specific NoPE property, with a RoPE counterfactual explaining why ArkVale cannot adopt it: (i) *build-once, never-stale row index* — scores are fixed bilinear forms of row content, so per-row entries computed at the compression event stay valid for the whole decode; under RoPE the effective key drifts with query–key distance, which is exactly why ArkVale must fall back to loose per-page bounding digests. (ii) *a free exact summand* — the never-rotated 64-dim branch makes $q_{\text{rope}} \cdot \text{side}_a$ an exact component of the true score at 11% of the row read; rotation leaves no exact sub-component to precompute. (iii) *a static certificate* — the sketch-truncation error is a per-row constant q_a only because the content form is fixed; a rotating subspace admits no static bound, hence similarity heuristics instead of certified admission. (iv) *one-shot calibration* — z and the gate threshold are calibrated once from the prefix and remain valid because score distributions are position-stationary; under RoPE prefix calibration drifts as the rotary bands sweep. (v) *zero positional fix-up on recall* — fetched rows splice into the same softmax with no re-rotation, which is what lets the stock kernel serve the merged partition unmodified.

Memory placement: no paging. Eviction here is a partition of *indices*, never a movement of data: in the standard (GPU-resident) configuration every row — kept and archived alike — stays in place in the one GPU pool, and a recall adds *indices* to the step’s attended set; the kernel gathers the rows where they lie. There is no page, no resident set, no replacement, and therefore no thrashing, by construction (the compression benefit in this configuration is bandwidth and compute, not VRAM). The host-offload variant does move archived rows to host memory once, at the compression event, off the critical path; a fire then costs a one-way, row-granular fetch (rows are immutable: no dirty state, no write-back, no displacement of anything resident), bounded under the cap by $\text{topj} \times H$ rows per step per layer. Neither configuration performs swap-in/swap-out in ArkVale’s sense.

The degeneration duality. The two designs fail in opposite directions, and the contrast is the sharpest way to state the difference. A fixed page budget never inflates per-step attention — ArkVale does *not* degenerate to naive MLA on recall — but under pressure it degrades *quality*:

pages beyond the budget are simply absent (plus transfer thrashing when the relevant set churns), and the only cure is inflating the budget toward the full cache, which un-does the compression. The certified competition degrades in the dual direction: uncapped, a pathological step can attend everything (naive MLA plus scan) while quality never degrades — the fetched rows are exact. The explicit cap is what turns this into an operator choice: quota designs hard-wire the degradation onto the quality side silently; here the side and the bound are stated.

3 Q3: What is the validation protocol?

1. **Kill-switch parity:** with the wrapper disabled, outputs must be call-for-call identical to stock. This isolates plumbing bugs from algorithm effects.
2. **Full-arm parity:** with selection forced to keep *all* rows, per-step outputs must match stock bit-for-bit. (The reference A/B harness switches arms on a live server this way; both arms then share one graph and code path, which is also the honest benchmark protocol.)
3. **Selector equivalence:** σ and the keep set must match the naive reference implementation — bit-exactly for the fallback path, decision-exactly (identical keep masks) for the fused kernels (engine/test/manual/test-vestigekv-equiv.py, self-contained).
4. **Quality gates** (uncapped, $\rho=1/32$): needle retrieval 1.00 at $8\times-32\times / 8k-65k$; teacher-forced Δbpb vs the uncompressed engine $\leq 0.15\%$; generation under the cap bit-identical.
5. **Recall-tier gates:** with the index built from the prefix (no training), recovery at 1/128 compression must reach 1.00 in the serving loop at 8k–32k, and the long-context necessity signature must reproduce: a high per-step trigger rate at $S\sim 512k$ with quality passing (the reference measures 99.8%).
6. **Scan-precision gates** (the 16-bit index): fire-set stability — the fired row set under the 16-bit operands must match the fp32 arm (Jaccard ≥ 0.99 , top-8 coverage unchanged), each arm calibrated independently, since the conformal threshold is refit after rounding — plus a serving-level greedy + needle A/B.
7. **Fusion parity:** every fused kernel keeps its pre-fusion implementation as a bit-parity oracle (fire/fetch sets and CSR contents identical on shared inputs); end-to-end, the fused pipeline’s 2048-token greedy output must be token-identical to the unfused one.
8. **Regression tests worth porting** (each caught a real bug): padded-batch refresh must not overrun the unpadded out_cache_loc; a re-used slot must not inherit the previous request’s kept state; the guard of each test must *fail* on the broken form (a check must be demonstrated to fail before it is trusted).

4 Q4: What defect classes did the port observe, and how were they mitigated?

1. **Stale per-slot state on slot reuse.** Runtime request slots are recycled. Any decode-side cache keyed by slot (the kept tables, tier-2 index) must be invalidated when a new request prefills the slot — otherwise the new request attends the *previous* request’s rows. This bug produced “compression enabled but attention time unchanged” and was only caught by a stage-separated kernel profile.

2. **Selection work inside the decode window.** Building indices lazily at the first decode step put $B \times L_{\text{MLA}}$ rFFTs into one step and erased the measured gain at $B=16$. All σ work belongs at prefill / block close (the frozen-selection corollary guarantees this is exact).
3. **Per-request host loops on the step path.** A Python-level loop of tiny copies (~ 100 ops/step at $B=16$) costs milliseconds; the vectorized pack replaces it with ~ 12 ops.
4. **Hidden device sync inside graph capture.** See the capture-safety rule in the whitepaper’s decode-path section.
5. **Padded-lane overruns.** See the padding trap in the same section; also give every padded lane a pad row, never an empty range.
6. **Arch-specific JIT of unrelated ops.** On new hardware, auxiliary ops (e.g. sampling kernels) may fail platform checks and kill the server at first generate — pin arch-agnostic fallbacks for everything outside the attention path before debugging the attention path.
7. **Graph-replay reality: a backend’s forward_* python does not run on the replay path.** Only the metadata hooks execute; anything a per-step algorithm needs must live in the fixed-address buffers those hooks can touch. Placing a per-step tensor copy inside the forward call either invalidates the graph (silent fall back to eager) or replays a stale capture-time value. The reference port measured the former: $138 \rightarrow 94$ tok/s from one such copy, with the profiler showing GPU busy 6.4ms against a 279ms step. Design the per-step data flow around this constraint before writing the recall path, not after.
8. **Isolate with a configuration matrix before attributing anything.** A performance investigation on this port spent ten successive hypotheses — each reasoned from the code, each microbenchmarked, each falsified — because the comparison itself was never validated first. The baseline had been measured on one upstream revision and the regression on another; the two were not comparable at all. Two cheap moves prevent this class of waste: reproduce the baseline on the *current* tree before calling anything a regression, and fill a small matrix (stock backend vs. wrapper-with-feature-off vs. wrapper-with-feature-on) $\times$ (old revision vs. new) so the failing cell names the variable. Record the upstream commit next to every serving number; never compare across a rebase.
9. **Host-side cost is invisible to isolated microbenchmarks.** Five hypotheses (scan GEMV, index pack, bookkeeping, extra attended rows, device-to-host syncs) were each measured in isolation, each came back cheap, and each was wrong about the whole: a single-call microbenchmark has no pipeline to stall. Only a stage-separated trace of the live server (busy-vs-gap) located the cost. Use microbenchmarks to price kernels, never to exonerate a host-side path.
10. **Benchmark hygiene.** Warm each graph bucket before measuring (first-use capture cost is large); interleave A/B arms on one live server; never compare across server restarts; measure with the serving framework’s own benchmark tool and state the one exception if any. Two corollaries the port learned the hard way: (a) the client’s inter-token timestamps degrade under stream batching above $\sim 240k$ context, so the server-side decode log is the per-token authority there; (b) launch-to-launch network variance ($\pm 15\%$ on the fixed terms) can invert a small cross-launch ratio — collect each arm’s curve within a single launch and read cross-arm claims as Δ -vs-4k, and never switch the baseline itself when reducing noise — a same-launch full-arm A/B removes launch variance but compares against a *different* (faster) baseline than the stock backend, which silently inflates the ratio.

11. **Second-graph launch tax (fixed, batch-independent).** An earlier build replayed the recall scan as a *second* CUDA graph per step; its host-side `cudaGraphLaunch` waited a full GIL slice and cost ~ 0.9 ms/step that did not scale with batch — enough to sink $bs=1$ below $1.0\times$ on its own. The fix is structural, not a knob: bake the whole recall step into the decode model graph, one launch per step. A knob-level attempt (shortening the GIL switch interval) helped the target microcost but regressed the process globally and was reverted; the lesson is that a per-step fixed cost is worth a structural fix, not a tuning band-aid.
12. **Capacity-grid launch floor.** The in-graph kernels launch a grid sized for the worst-case context and batch, so at small live batch the placeholder pairs cost real launch time even though they compute nothing. Two mitigations, both content-independent: give each program a whole 1024-row compact bucket (a $16\times$ smaller grid than one-block programs), and clip the grid per batch-size class to the live pair count so placeholders are not launched at all. What remains is the platform’s own per-batch-class graph overhead, which the dense arm pays too.